# Reflex Computation: Hard-Arithmetic Scaling and Semantic Execution Blocks

Working Draft

August 2026

**Abstract**

Language models can describe arithmetic procedures while still making exact execution errors. Conventional tool-use systems address this weakness by requiring the model to decide to call a calculator and serialize an invocation. We study bounded local calculator interfaces that act during generation. The original strict watcher recognizes an arithmetic suffix before an equals sign; the next iteration adds deterministic LaTeX/Unicode surface normalization and a fenced calculator block that executes only when its closing delimiter arrives. All assisted conditions share the same typed postfix micro-IR, exact-rational engine, resource bounds, and textual reinjection. After baseline-only adaptive selection, an XPU comparison used 16 arithmetic examples, 12 controls, and seven conditions. Exact-match accuracy was 43.8% for LLM-only, 62.5% for explicit tools, 75.0% for normalized reflex, and 12.5% for the original zero-shot block; normalized reflex gained five paired examples and lost none ($p = 0.0625$, unadjusted exact McNemar). We then developed block prompts on a separate split and froze a negative-first, positive-last two-shot protocol. On the same diagnostic examples, frozen blocks reached 100% valid execution and arithmetic accuracy with Qwen3-0.6B, but falsely blocked 2 of 12 controls. Qwen3-1.7B reached 93.8% execution and accuracy with zero false blocks, gaining nine and losing one pair against LLM-only ($p = 0.0215$). A Qwen3-4B six-item XPU smoke reached 100% execution with no false block, but is not a held-out estimate. These small, developmental matched-family results show that semantic execution blocks can be an ICL interface skill; they motivate a minimal small-model protocol adapter and a powered confirmatory set, not a model-family or general scaling claim.

## 1 Status and Scope

This document reports an implemented next-iteration protocol, the preliminary pilot that motivated it, adaptive calibration, and one held-out comparison. The repository contains a dependency-free calculator/runtime path, deterministic dataset generation, JSONL traces and summaries, a scripted plumbing check, and an optional token-by-token Hugging Face adapter. Scripted checks are marked `empirical: false`; their expected perfect calculator behavior must not be cited as model evidence. The XPU pilot reported below is empirical but too small for model-family, scaling, or replicated performance claims. The held-out run is empirical and protocol-frozen, but it also has one checkpoint and one greedy seed; its condition ordering is evidence for replication, not a final architecture-level estimate.

Paper 1 asks one question:

> When a language model naturally exposes a completed arithmetic expression during generation, does automatic bounded evaluation improve reliability or cost relative to model-only generation and explicit calculator calls?

The scope remains calculator only and deterministic syntax interfaces. The experiment does not test semantic recognition of prose arithmetic, learned routing, persistent symbolic memory, transactional state, Progressive Retrieval Attention (PRA), or native key/value access. Those mechanisms would make a positive result harder to attribute and a negative result harder to diagnose.

## 2 Hypotheses

**H1: exact reliability.** On arithmetic examples for which the strict event is correctly detected and normalized, reflex assistance increases exact-match accuracy relative to the same model without assistance.

**H2: interface efficiency.** Reflex or fenced-block assistance reduces model-authored invocation overhead relative to an explicit calculator condition while remaining competitive in answer quality and wall time.

**H3: difficulty scaling.** As operator count and operand width increase, reflex accuracy degrades more slowly than model-only and matched-prompt conditions. The relevant evidence is the condition-by-difficulty interaction or fitted scaling slope, not one easy benchmark cell.

**H4: model-size leverage.** Smaller models receive at least as much marginal accuracy benefit from correct reflex assistance as larger matched-family models. This is tested as a model-size-by-condition interaction, not inferred from separate unrelated checkpoints.

These hypotheses are conditional on detection and use. Oracle detection estimates interface headroom; engine correctness alone does not demonstrate that the model will expose the syntax or use the inserted result.

## 3 Mechanism

### 3.1 Strict generation-time event

The reflex recognizes an integer arithmetic suffix completed by one equals sign, for example

```
The total is 37 * 48 =
```

and inserts the exact result before later generated text:

```
The total is 37 * 48 = 1776.
```

The event is ordinary written arithmetic, not a bespoke API token. Detection is incremental at character granularity even when a tokenizer supplies multi-character fragments. Thus, if an equals sign occurs in the middle of a fragment, the result is inserted before the remainder of that fragment is processed.

The lexical detector accepts digits, whitespace, parentheses, and `+`, `-`, `*`, `/`, `//`, `%`, and `**`. It requires an operator-like symbol, balanced parentheses, and a digit or closing parenthesis before the equals sign. It suppresses double-quoted candidates and rejects candidates adjacent to letters, underscores, or decimal points. Lexical detection remains separate from normalization: a candidate such as `+ 2` is logged and then rejected because it has no binary operation.

Table 1: Default calculator safety and complexity bounds.

| Bound | Default |
|---|---|
| Expression characters | 256 |
| AST nodes / depth | 64 / 16 |
| Digits per integer literal | 64 |
| Absolute integer exponent | 12 |
| Numerator or denominator size | 512 bits |
| Stack items | 64 |
| Engine time budget | 25 ms |
| Recognizer suffix buffer | 512 characters |

### 3.2 Typed micro-IR

The normalizer parses the candidate in expression mode and accepts only integer constants, binary operations, unary plus, and unary minus. Calls, names, attributes, indexing, containers, comparisons, booleans, floats, and all other syntax are rejected. The parsed tree is converted to a canonical postfix instruction sequence such as

`PUSH 7; PUSH 5; ADD; PUSH 3; MUL.`

The Python parser is used only to construct and inspect syntax. The source is never compiled or executed. A separate stack interpreter executes the micro-IR using exact rational numbers.

### 3.3 Resource policy

Every event is bounded before or during evaluation. The default policy is shown in Table 1. Bounds are configuration recorded in each request, not hidden assumptions.

Division by zero, malformed IR, timeout, and resource exhaustion are typed failures. Failed results are traced but not inserted. Exponent and intermediate bit limits are checked before a large value can propagate through subsequent instructions.

### 3.4 Observable interrupt pipeline

Each intervention follows

DETECT → NORMALIZE → ROUTE → EXECUTE → STATE UPDATE → REINJECT.

Every transition receives a run ID, sequence number, timestamp, status, candidate/request IDs, duration where meaningful, and structured details. The append-only micro-state stores the typed request, result, creation time, detector, and engine provenance. This is enough to separate detector, parser, engine, and reinjection failures without importing the later transactional-state design.

## 4 Conditions

All conditions use the same arithmetic examples, checkpoint revision, decoding budget, and calculator implementation when a calculator is available.

The explicit format is deliberately unambiguous and is not counted as a reflex. Its parser and the reflex parser converge on the same arithmetic micro-IR and engine, isolating invocation style from calculator capability. A stronger native function-calling baseline should be added when

Table 2: Minimum experimental conditions.

| Condition | Model instruction and capability | Purpose |
|---|---|---|
| LLM only | Return the exact answer; no calculator | Unassisted checkpoint baseline |
| Matched prompt | Work carefully and check operations; no calculator | Stronger prompting without external capability |
| Explicit tool | Model writes `<tool:calculator>EXPR</tool>` and receives the same engine result | Conventional model-authored invocation baseline |
| Reflex | Model is asked to write the expression followed by one equals sign; runtime detects any valid strict suffix | Proposed automatic condition |
| Normalized reflex | Reflex syntax plus deterministic LaTeX, Unicode, whitespace, and bracket normalization | Surface-robust automatic condition |
| Calculator block | Model writes one fenced calculator block; execution waits for the closing fence | Explicit boundary without an API schema |
| Oracle | Same generation instruction as reflex, but the detector accepts only the gold canonical expression | Detection/selection headroom control |

the selected model/runtime exposes a stable function-calling interface; the textual baseline is the portable minimum.

# 5 Benchmark

## 5.1 Factorial arithmetic generators

The legacy deterministic generator crosses operator count with operand digit width. The separate `hard_arithmetic_v1` generator crosses at least four operator-count bands, four digit-width bands, and three structures: left chains, nested trees, and multiplication trees. It includes addition, subtraction, multiplication, safe exact-rational division, mixed operations, and long intermediates. Each cell uses identity-derived random seeds, and gold answers are produced by a separate exact-rational reference evaluator rather than the calculator implementation under test. The full hard grid contains

$$4 \text{ operator counts} \times 4 \text{ digit widths} \times 3 \text{ structures} = 48 \text{ cells}$$

before replicate and seed factors. Every generated item is accepted only after the independent oracle and bounded calculator agree.

This synthetic benchmark isolates scaling and component correctness; it does not establish ecological prevalence. A later extension may add held-out natural model traces, but only after freezing annotation rules for whether a generated prefix contains a valid strict event.

## 5.2 Non-trigger and adversarial controls

Controls include prose containing numbers, ranges, parentheses, quoted arithmetic, decimal-looking versions, variable equations, and code-like equality. Additional engine tests cover division by zero, excessive exponents and values, disallowed calls/names/indexing, and unary-only candidates. Controls are scored for false intervention and component behavior, not arithmetic answer accuracy.

### 5.3 Model and seed sweep

The primary manifest pins one matched Qwen3 family at 0.6B, 1.7B, and 4B parameters and uses seeds 17, 29, and 41. Repository commit hashes are stored in the configuration and copied into run manifests. The model list is an initial controlled family, not a claim of model-family generality. A second family is required before making architecture-general claims.

Generation is greedy in the initial adapter so seed variability mainly reflects future sampled configurations and experimental plumbing; if sampling is enabled, temperature, top-$p$, and all random states must be pinned. The correctness-first adapter recomputes the full prefix per generated token after reinjection. This is intentionally inefficient but semantically transparent. Optimized KV-cache interception is outside Paper 1 and must not be silently credited to it.

## 6 Metrics and Analysis

**End task.** Report exact rational answer accuracy with 95% confidence intervals. If a model states a final answer after receiving a calculator result, score the final claim, not merely the correct engine value. Report correction, override, and result-use rates separately.

**Components.** Report intended-expression exposure, candidate recognition, full-expression selection, normalization acceptance/correctness, engine success/correctness, reinjection success, result use, and later override separately. Also report false-intervention rate, intervention count, and state-item count. Oracle differences localize missed or malformed interface events.

**Cost.** Report prompt, model-generated, and reinjected tokens separately; model calls; engine and end-to-end wall time; and serialized trace/state growth. Do not present the correctness-first Hugging Face adapter's full-prefix recomputation as a production latency estimate.

**Scaling and inference.** Report every operator-count by digit-width cell for each model, condition, and seed. Estimate condition-by-difficulty and condition-by-model-size interactions, with paired comparisons because all conditions share examples. Predeclare one primary endpoint and correct or label exploratory multiple comparisons. Preserve raw predictions and event traces so component errors can be reclassified.

## 7 Frozen Next-Iteration Protocol

The current interface freeze is versioned as `paper1_hard_interfaces_v2_concise`. It retains `strict_arithmetic_v1` unchanged and adds `normalized_arithmetic_v1`, `surface_normalizer_v1`, and `calculator_block_v1`. The normalizer maps allowlisted multiplication, division, Unicode-minus, bracket, and spacing aliases before delegating to the existing `ccpu.arithmetic.postfix.v1` parser. Unknown commands, prose, variables, quoted arithmetic, and quoted or inline fence examples remain inert.

The block grammar is a line-anchored Markdown-like fence named `calculator`. Its contents are collected incrementally, but no candidate is emitted until the closing fence is complete. Thus a valid intermediate line cannot execute prematurely. At close, the same surface normalizer and bounded calculator are used, and a compact typed result is inserted.

The adaptive pilot runs only the LLM-only condition over all 48 cells. Cell selection uses aggregate exact match to choose regimes near 95%, 65%, 30%, and 0%; it never examines assisted

outcomes. Once those cells are selected, their definitions, prompts, grammar, normalizer, and diagnostics remain frozen. A distinct dataset seed then generates the held-out comparison across all seven conditions.

Every arithmetic prediction records expression exposure, candidate recognition, full-expression selection, normalization, execution, reinjection, result use, and later override independently. Cost records include prompt, generated, and reinjected tokens, model calls, wall and engine time, serialized trace/state bytes, and deterministic invocation-delimiter characters. Scripted smoke output is marked `empirical: false` and is plumbing evidence only.

### 7.1 Adaptive calibration status

Two baseline-only XPU calibrations exposed output censoring before any assisted condition was run. Both used Qwen3-0.6B, 96 arithmetic examples over 48 cells, two nominal generation seeds, and a 48-token ceiling, for 192 generations each. Under the original prompt, all 192 generations reached the ceiling and 184 had no scorable final answer. A uniformly applied concise-answer prompt reduced cap hits to 170 and missing answers to 156, but both versions yielded only 3.125% overall exact match per seed. The concise run had 46 cells at 0%, one at 50%, and one at 100%, so it cannot supply the four intended difficulty bands.

These runs are preserved as empirical calibration failures, not treated as the hard-arithmetic comparison. Since greedy argmax decoding produced identical text for every one of the 96 cross-seed pairs, the revised calibration removes the redundant generation seed, broadens the grid downward to one–four operand digits and one–four operators, uses two independently generated examples per cell, and raises the uniform ceiling to 96 tokens.

The revised calibration completed 96 baseline-only generations with 30.2% overall exact match; 78 generations had a scorable final answer. Across 48 cells, 9 reached 100% accuracy, 11 reached 50%, and 28 reached 0%. We froze four whole cells with complete answer coverage and no cap hits: one operator/one digit with a left chain (100%), two operators/two digits with nesting (50%), one operator/three digits with a multiplication tree (50%), and one operator/four digits with a left chain (0%). The two 50% cells are explicitly coarse approximations to the overlapping moderate and transition targets.

The selection record stores source dataset and prediction hashes and states that no assisted outcome was consulted. A separately seeded held-out dataset uses four examples per selected cell, cycling ASCII, LaTeX, Unicode, and bracket surfaces, plus all 12 adversarial controls. The held-out model run remains ungenerated at this protocol-freeze point; the results below were produced only after that dataset, selection record, and interface code were committed.

## 8 Held-Out XPU Comparison

### 8.1 Setup and end-task result

The held-out run used Qwen3-0.6B at revision `c1899de`, PyTorch 2.13.0+XPU, one greedy generation seed (17), and a uniform 96-token ceiling. Four newly generated examples in each of the four frozen cells gave 16 arithmetic examples; 12 controls were crossed with the same seven conditions. All 196 prediction rows report `empirical: true` and `device: xpu`.

Normalized reflex was numerically the strongest non-oracle condition, improving by 31.25 percentage points over LLM-only and 12.5 points over explicit tools. The five paired gains and zero losses are suggestive but do not reach 0.05 in the unadjusted exact test; no multiplicity correction is applied. Strict reflex lost one paired example and gained none. The four held-out cells contain only

Table 3: Held-out exact-match accuracy. Intervals are 95% Wilson intervals over 16 arithmetic examples. Paired gains/losses and unadjusted exact McNemar $p$ values compare each condition with LLM-only.

| Condition | Accuracy [95% CI] | Gains–losses | Exact $p$ |
|---|---|---|---|
| LLM only | 0.438 [0.231, 0.668] | — | — |
| Matched prompt | 0.500 [0.280, 0.720] | 1–0 | 1.000 |
| Explicit tool | 0.625 [0.386, 0.815] | 3–0 | 0.250 |
| Strict reflex | 0.375 [0.185, 0.614] | 0–1 | 1.000 |
| Normalized reflex | 0.750 [0.505, 0.898] | 5–0 | 0.0625 |
| Calculator block | 0.125 [0.035, 0.360] | 2–7 | 0.180 |
| Oracle | 0.875 [0.640, 0.965] | 7–0 | 0.0156 |

Table 4: Held-out assisted-interface diagnostics over arithmetic examples. Selection is correctness conditional on an accepted request; use is conditional on successful execution. All conditions had zero false interventions on 12 controls, and engine correctness was 1.0 whenever executed.

| Condition | Exposure | Recognition | Selection | Normalize/execute | Use | Override |
|---|---|---|---|---|---|---|
| Explicit tool | 0.875 | 1.000 | 1.000 | 0.625 | 1.000 | 0.000 |
| Strict reflex | 0.625 | 0.313 | 0.600 | 0.313 | 0.800 | 0.200 |
| Normalized reflex | 0.750 | 0.750 | 1.000 | 0.750 | 1.000 | 0.000 |
| Calculator block | 0.375 | 1.000 | 1.000 | 0.125 | 1.000 | 0.000 |
| Oracle | — | 1.000 | 1.000 | 1.000 | 0.875 | 0.125 |

four examples each and do not show a stable monotone difficulty interaction: normalized-reflex cell accuracies were 1.00, 0.25, 1.00, and 0.75. H1 is therefore unsupported for the original strict watcher, while the normalized variant is a replication candidate rather than a confirmed claim.

### 8.2 Interface decomposition

Normalized reflex reached and correctly selected the full expression in 12 of 16 cases; all 12 exact results were reinjected, used, and not overridden. Strict reflex executed five candidates, but only three represented the intended full expression. Explicit tool calls were recognized in every arithmetic generation, but six arguments failed the strict parser, primarily because the model copied LaTeX or Unicode notation.

The semantic block result falsifies H3 for the tested zero-shot prompt. All 16 closing fences were recognized, but 14 blocks contained the literal placeholder `EXPRESSION` or mixed instructions/prose with arithmetic. Only two blocks normalized and executed; both engine results were exact and used. Thus the block boundary prevented premature execution, but its demonstration-style prompt elicited invalid block contents. This is a generation/normalization failure, not a calculator or closing-fence detection failure.

**Cost.** Mean generated tokens were 43.1 (LLM-only), 46.1 (matched), 19.2 (explicit), 47.9 (strict), 22.4 (normalized), 32.7 (block), and 15.6 (oracle). Mean wall times followed the correctness-first decoder rather than production serving: 8.0, 8.8, 3.2, 9.6, 4.0, 5.9, and 2.7 seconds, respectively. Normalized reflex used fewer delimiter characters than explicit calls, but it did not beat explicit tools on every measured cost: explicit generated fewer tokens and was faster. H2 is therefore not supported as a quality–cost frontier claim.

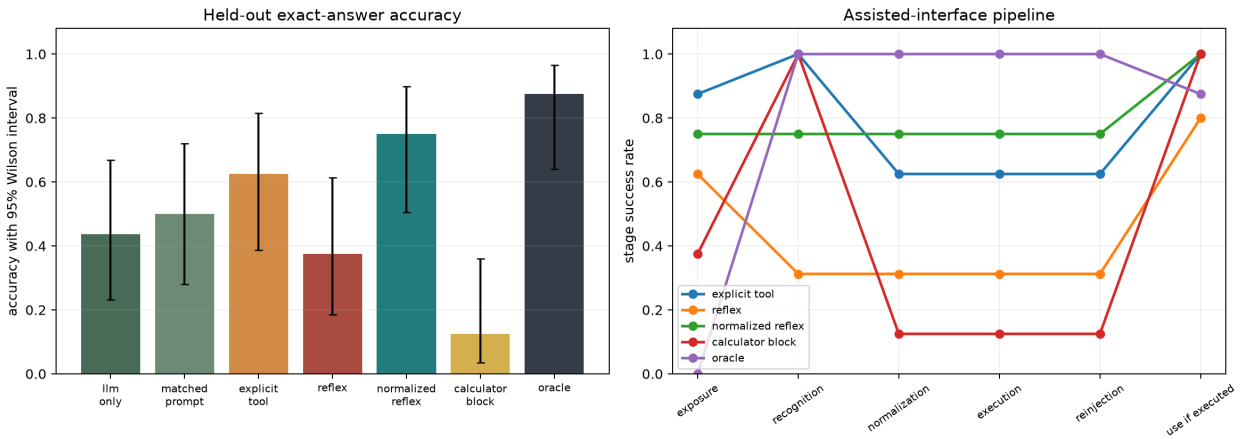


Figure 1: Held-out accuracy (left) and assisted-interface stage decomposition (right). Exposure is model-stream exposure and is not applicable to oracle priming. Use is conditional on execution, which explains high use despite low block normalization.

### 8.3 Initial evidence-gate decision

At that point the decision was **no-go for Paper 2**. The original strict reflex does not improve held-out accuracy, the block interface fails zero-shot content elicitation, and the normalized-reflex advantage is unreplicated with $p = 0.0625$ over 16 pairs. A next Paper 1 replication should hold `surface_normalizer_v1` fixed, increase examples and generation seeds under sampling or add a second checkpoint family, and predeclare normalized reflex versus both LLM-only and explicit tool as primary comparisons. Block prompting or lightweight training is now eligible for a separate development split, but must not be tuned on this held-out set.

## 9 Few-Shot Blocks and Capability Diagnostic

### 9.1 Endpoint audit and prompt development

The original exact-match labels are preserved. A parallel, condition-independent endpoint extractor recognizes terminal forms such as `Response: N` without using condition identity. Under this audit the original zero-shot block's answer accuracy changes from 12.5% to 43.8%, while normalized reflex remains 75.0%. This is an extraction correction only: valid zero-shot block execution remains 12.5%, so the protocol conclusion is unchanged.

We tested six stronger block prompts on a separate developmental split containing four arithmetic examples and four controls. Concrete and verbatim-copy examples made arithmetic execution reliable but initially caused false blocks on all four controls; a positive/negative variant suppressed false blocks but also suppressed arithmetic blocks. A diagnostic with fully seeded fences isolated payload copying. The final order control, ICL-G, puts a negative demonstration first and a concrete positive block last. It achieved 100% semantic payload equivalence, execution, result use, and answer accuracy with no developmental false block. Exact-string payload match was 0% because the model removed redundant parentheses; the bounded parser established semantic equivalence. We froze this prompt as `paper1_calculator_block_icl_v2_order_control` before cross-model runs.

Table 5: Frozen ICL-G calculator blocks. The 0.6B and 1.7B rows use 16 arithmetic and 12 control examples; the 4B row is a four-arithmetic/two-control smoke only.

| Model | Phase | Execute | Semantic | Answer | False controls | Use |
|---|---|---|---|---|---|---|
| Qwen3-0.6B | held-out diagnostic | 1.000 | 1.000 | 1.000 | 2/12 | 1.000 |
| Qwen3-1.7B | held-out diagnostic | 0.938 | 0.938 | 0.938 | 0/12 | 1.000 |
| Qwen3-4B | smoke | 1.000 | 1.000 | 1.000 | 0/2 | 1.000 |

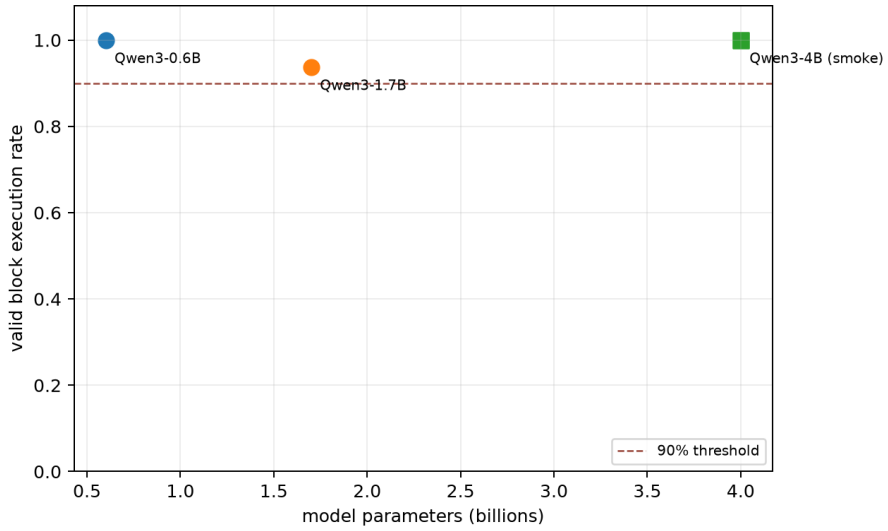


Figure 2: Valid execution under the one frozen ICL prompt. Circles are 16-example held-out diagnostics; the square is a four-example smoke and must not be read as a comparable estimate. Parameter count is descriptive, not a fitted scaling law.

### 9.2 Matched-family XPU results

Table 5 compares the frozen prompt without per-model tuning. Qwen3-0.6B used the original held-out arithmetic/control set. The 1.7B checkpoint used all seven conditions over the same 28 examples, producing 196 rows. The 4B result is only the predeclared four-arithmetic/two-control smoke gate.

For 0.6B, ICL-G gained nine arithmetic examples and lost none against the preserved LLM-only run ($p = 0.0039$); however, two false blocks fail the safety part of the frozen gate. For 1.7B, it gained nine and lost one ($p = 0.0215$), while normalized reflex gained six and lost one ($p = 0.125$). The sole 1.7B block failure omitted the wrapper and emitted a bare expression. The endpoint audit leaves ICL-G at 93.8%. The unexpectedly low 1.7B oracle score (12.5% originally, 25.0% under endpoint rescoring) reflects model integration after seeded results, not calculator failure; it is retained rather than normalized away.

**Family coverage and runtime.** Official Llama 3.2 and Gemma checkpoints require manually accepted Hugging Face licenses. No authenticated token was available, so four configured checkpoints were skipped and no community mirror was substituted. The current evidence is therefore Qwen-only. Qwen3-1.7B peaked at 3.57 GB XPU memory and its full run took 3074 seconds. Qwen3-4B loaded and completed the smoke at 8.21 GB peak memory in 494 seconds. The block, accuracy/interface, token–latency, and failure plots are generated from one hashed comparison

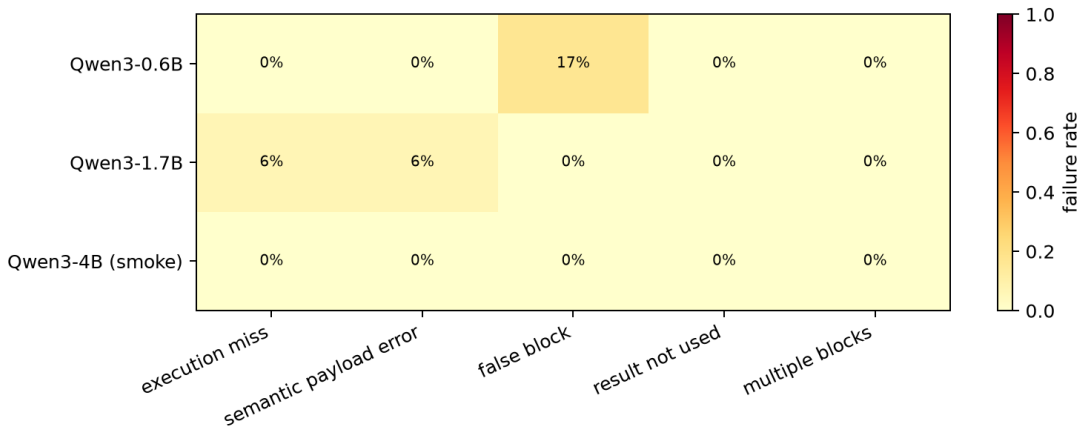


Figure 3: Block failure decomposition. The small model's remaining error is control discrimination, whereas the 1.7B model has one arithmetic protocol miss.

configuration and machine-readable table.

### 9.3 Current evidence and LoRA gate

The new result reopens the original interface gate: a frozen semantic block is reliable on the 1.7B diagnostic and clean in the 4B smoke. This is sufficient to motivate later heterogeneous-interface work, while confirmatory Paper 1 claims remain gated on a larger untouched set and additional families. It also places the LoRA question in Regime A: can 0.6B retain its perfect arithmetic protocol while eliminating false control blocks? A minimal protocol-only Qwen3-0.6B adapter is justified as the next experiment, but was not trained here. The validated XPU environment lacks PEFT/TRL/Accelerate, and modifying it would mix an unvalidated training stack into the completed inference environment. Any adapter result must use leakage-audited expression/control seeds, report training tokens/time and parameter count, and compare base+ICL, adapter+ICL, and adapter+minimal instruction. No present result supports a claim that adaptation learned arithmetic.

## 10 Preliminary XPU Pilot

### 10.1 Setup

We ran a diagnostic pilot with Qwen3-0.6B at pinned revision `c1899de` (the full hash is stored in the manifest), PyTorch 2.13.0+XPU, and an Intel integrated GPU. The deterministic sample crossed one and three operators with one- and two-digit operands, using addition and multiplication. It contained two examples per cell (eight arithmetic examples) and eight matched non-trigger controls. One generation seed (17) and five conditions yielded 80 generations. The 96-token output budget was selected after a shorter plumbing run exposed truncation; the shorter run is not included in the results. Confidence intervals are Wilson intervals over only eight arithmetic observations per condition and are therefore intentionally wide.

Table 6: Exploratory XPU pilot. Accuracy intervals are 95% Wilson intervals; tokens and latency are means. Latency reflects the correctness-first full-prefix adapter and is not a production estimate.

| Condition | Accuracy | Trigger recall | Output tokens | Latency (ms) |
|---|---|---|---|---|
| LLM only | 0.750 [0.409, 0.929] | — | 40.56 | 8028 |
| Matched prompt | 0.625 [0.306, 0.863] | — | 45.81 | 9380 |
| Explicit tool | 0.875 [0.529, 0.978] | 0.625 | 22.50 | 4410 |
| Reflex | 0.750 [0.409, 0.929] | 0.750 | 46.06 | 9195 |
| Oracle | 1.000 [0.676, 1.000] | 1.000 | 15.31 | 2718 |

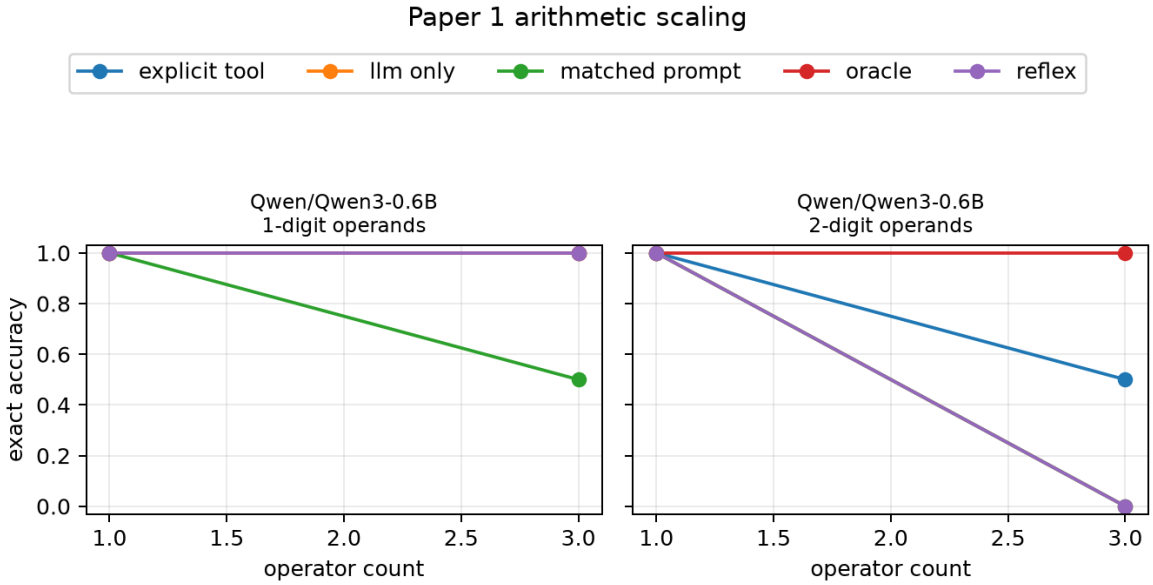


Figure 4: Exploratory exact-match accuracy by generated difficulty cell. Each point contains only two examples; the figure diagnoses plumbing and candidate behavior and must not be read as a scaling estimate.

### 10.2 Results

The reflex condition did not improve exact-match accuracy over LLM only in this pilot. Explicit-tool accuracy was numerically higher and oracle intervention was perfect, but the sample is too small for either difference to support a general claim. No condition falsely intervened on a control. Accepted explicit-tool candidates represented the intended expression in every triggered case, whereas only 50% of reflex candidates did. Inspection showed that the strict watcher often intercepted a valid intermediate subexpression rather than the full task expression. Once a candidate reached the engine, execution correctness was 100% in both conditions. Reflex result-use was 66.7%, and one third of used results were later overridden. These component results locate the present bottleneck in exposure, selection, and downstream use rather than arithmetic execution.

At this protocol revision H1 is not supported: the automatic reflex tied the unassisted baseline and failed to dominate explicit invocation. H2–H4 remain untested because this pilot includes neither a production interface nor the required model-size and seed sweeps. Paper 2 therefore remains gated. A confirmatory run must increase examples per cell, use all declared seeds and model sizes, add a second family, and freeze any detector change before examining outcomes.

## 11 Falsification and Evidence Gate

The reflex claim is not supported if any of the following holds under matched capability and declared cost accounting:

- answer gains disappear when final model claims, rather than inserted engine values, are scored;
- false triggers, normalization errors, or ignored results offset exact execution gains;
- explicit calculator calling matches or dominates the quality–token–latency frontier;
- reflex and baseline accuracy have indistinguishable difficulty slopes;
- gains occur only with oracle detection or only in one easy/model cell; or
- results fail to reproduce across seeds or a second model family.

Paper 2 is justified only if Paper 1 identifies a reproducible regime where automatic strict assistance helps and mixed operation types expose a limitation of one calculator. Paper 3 is not justified by syntax brittleness alone; learned interrupts require a measured task regime worth extending.

## 12 Reproduction

The reusable runtime lives under `src/ccpu/common`; Paper 1 code lives under `src/ccpu/paper1`. From the repository root, the dependency-free protocol check is:

```
python -m pip install -e .
python -m pytest
python -m ccpu paper1 generate \
  --config configs/paper1/smoke.json \
  --output artifacts/paper1/smoke/dataset.jsonl
python -m ccpu paper1 validate \
  --dataset artifacts/paper1/smoke/dataset.jsonl
python -m ccpu paper1 simulate \
  --dataset artifacts/paper1/smoke/dataset.jsonl \
  --output-dir artifacts/paper1/smoke/run
```

The pinned pretrained path is:

```
python -m pip install -e ".[hf]"
python -m ccpu paper1 generate \
  --config configs/paper1/primary.json \
  --output artifacts/paper1/primary/dataset.jsonl
python -m ccpu paper1 run-hf \
  --dataset artifacts/paper1/primary/dataset.jsonl \
  --config configs/paper1/primary.json \
  --output-dir artifacts/paper1/primary/run
```

The reported XPU pilot uses the direct interpreter from the validated XPU environment and can be reproduced with:

```
$py = 'C:\Users\j.simao\.venvs\modal-llm-xpu\Scripts\python.exe'
$env:PYTHONPATH = (Resolve-Path src).Path
& $py -m ccpu paper1 generate \
  --config configs/paper1/xpu_pilot.json \
  --output artifacts/paper1/xpu_pilot/dataset.jsonl
& $py -m ccpu paper1 run-hf \
  --dataset artifacts/paper1/xpu_pilot/dataset.jsonl \
  --config configs/paper1/xpu_pilot.json \
  --output-dir artifacts/paper1/xpu_pilot/run96
& $py -m ccpu paper1 replay \
  --dataset artifacts/paper1/xpu_pilot/dataset.jsonl \
  --completions artifacts/paper1/xpu_pilot/run96/predictions.jsonl \
  --output-dir artifacts/paper1/xpu_pilot/final
```

The held-out comparison and derived analysis use:

```
& $py -m ccpu paper1 run-hf \
  --dataset artifacts/paper1/hard_heldout_xpu/dataset.jsonl \
  --config configs/paper1/hard_heldout_xpu.json \
  --output-dir artifacts/paper1/hard_heldout_xpu/run \
  --device xpu
& $py -m ccpu paper1 paired \
  --dataset artifacts/paper1/hard_heldout_xpu/dataset.jsonl \
  --predictions artifacts/paper1/hard_heldout_xpu/run/predictions.jsonl \
  --output artifacts/paper1/hard_heldout_xpu/run/paired_analysis.json
python -m ccpu paper1 plot-interfaces \
  --summary artifacts/paper1/hard_heldout_xpu/run/summary.json \
  --output docs/papers/paper1/paper1_heldout_interfaces.png
```

Device selection for `device: auto` is CUDA, then XPU, then CPU. The replay step applies the current frozen evaluator to preserved generations and records both the source prediction hash and rescore provenance.

Use `--limit`, `--model`, and repeated `--condition` arguments for resource preflight. Each run writes predictions, full traces, summary tables, hashes, configuration, model metadata, environment information, and git revision/dirty status. The replay command permits model generation in a separate harness while preserving identical controller and evaluation semantics. The `plot` command renders the machine-readable scaling cells as accuracy curves by model, operand width, condition, and operator count. The `compare-models` command consumes `configs/paper1/block_icl_comparison.json` and regenerates the cross-model table plus block-capability, interface-accuracy, token–latency, and failure-mode figures without manually entered results.

## 13 Limitations

The elicitation instruction asks reflex models to expose an expression followed by an equals sign. This is less explicit than an API call but still changes generation behavior, and a model that answers directly may never trigger. Both trigger rate and conditional-on-trigger accuracy are therefore necessary. The strict lexical grammar does not understand intent and will miss prose

arithmetic; expanding semantics belongs to Paper 3. Quote suppression is deliberately limited, and code-like or unusual formatting remains an adversarial surface.

The synthetic task controls arithmetic complexity but not realistic discourse. The textual explicit-tool baseline may understate a model's native structured function-calling ability. Full-prefix token recomputation overstates runtime cost. The main comparisons have one greedy seed, 16 arithmetic examples, and four observations per selected cell. Their intervals remain wide, the adaptive bands are coarse, and no architecture or universal scaling generalization is supported. The 4B row is only a six-item smoke. Llama and Gemma were unavailable behind manual authorization gates, leaving family effects unresolved. Twelve controls do not tightly bound rare false triggers, as the two 0.6B ICL-G failures illustrate. The zero-shot block prompt included a placeholder demonstration that the model often copied literally; its historical result is not reinterpreted by the separately developed ICL prompt or endpoint audit. Finally, exact calculator output cannot repair an incorrectly copied expression, a model that ignores the result, or a task whose premises are wrong.

## 14 Related Work

Toolformer trains language models to decide when and how to invoke APIs, including a calculator [1]. PAL delegates model-generated programs to an interpreter [2], while ReAct interleaves model-authored reasoning and actions [3]. These systems establish the value of external execution. Paper 1 isolates a different control point: a runtime recognizes ordinary strict arithmetic already emerging in the output and invokes a bounded engine without a function-name decision. The comparison to explicit invocation is empirical; the paper does not assume that the reflex interface is better.

## 15 Conclusion

Paper 1 reduces the cognitive-coprocessor proposal to its smallest credible experiment. A character-incremental detector, typed allowlisted micro-IR, exact bounded calculator, append-only state, and immediate text reinjection are enough to test whether automatic computation can improve an autoregressive stream. The original strict watcher does not improve accuracy, and the zero-shot fenced block usually emits invalid content. Deterministic surface normalization is numerically stronger, executing 12 of 16 full expressions and improving five paired examples without a loss, but remains an unreplicated result. Separate development shows that the block failure was not intrinsic to the grammar: one frozen two-shot prompt reaches 100% arithmetic execution on Qwen3-0.6B and 93.8% on Qwen3-1.7B. Capability changes the error type: 0.6B falsely blocks two controls, while 1.7B has no false block and one arithmetic miss. A clean 4B smoke supports feasibility but not scaling. Paper 1 therefore establishes a plausible few-shot semantic execution interface within one model family, reopens the gate for bounded multi-engine work, and motivates a protocol-only small-model adapter. Powered confirmatory data, authorized second-family runs, and a separately validated LoRA training stack remain necessary before stronger claims.

## References

[1] T. Schick, J. Dwivedi-Yu, R. Dessi, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom. Toolformer: Language models can teach themselves to use tools. *NeurIPS*, 2023. https://arxiv.org/abs/2302.04761.

[2] L. Gao, A. Madaan, S. Zhou, U. Alon, P. Liu, Y. Yang, J. Callan, and G. Neubig. PAL: Program-aided language models. *ICML*, 2023. https://arxiv.org/abs/2211.10435.

[3] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. *ICLR*, 2023. https://arxiv.org/abs/2210.03629.